

8) Dynamic Search

connect(ui->lineEdit, &QLineEdit::textChanged, this->GUI::radiusChanged)

13) QSlider connect + fct.

connect(ui->slider, &QSlider::valueChanged, this->GUI::radiusChanged)
 void GUI::radiusChanged(int value) {
 radius = value;
 populateList();
 }

Test stuff I fear the most (face my fears w me)

8) Dynamic Search

- set its properties in the main window (GUI)
 - min (1)
 - max (50)
 - value (10)
- connect(ui->slider, &QSlider::valueChanged, this->GUI::radiusChanged)
- void GUI::radiusChanged(int val){
 - radius = val;
 - populateList;
- radius is a public field in the GUI.
- retrieve value from a slider:
 - int val = ui->slider->value();
- set value of a slider:
 - ui->slider->setValue(10);

slider.

set a layout

n

show?

'save'?

2) QSpinBoxes

- in constructor: ui->spinBox->setEnabled(false)
- connect(ui->spinBox, QOverload<int>::of(&QSpinBox::valueChanged), ...)
- when I select sth in the list:
 - ui->spinBox->blockSignals(true);
 - ui->spinBox->setValue(a.get...());
 - ui->spinBox->blockSignals(false);
 - ui->spinBox->setEnabled(condition);
- retrieve value from a spinbox:
 - ui->spinBox->value();

Qt doc??

self;
 ui->comboBox
 "All" ||
 std::string(e...
 set.push_back
 All" return
 ;
 (date | Sa

3) open new window from GUI

QWidget* window = new QWidget();

~ just like GUI* gui = new GUI() ... in main()

4) if you want to type in that window -> QVBoxLayout

```
auto layout = new QVBoxLayout(window);
for (...) {
    auto textEdit = new QTextEdit();
    textEdit->setText(QString::fromStdString(...));
    auto button = new QPushButton("save");
    layout->addWidget(textEdit);
    layout->addWidget(button);
    connect(button, &QPushButton::clicked, this, [this, textEdit, id] {
        save(...);
    });
}
```

capture list

review
Screenwriting

5) TableModel

✓ inside domain folder → TableModel.

h

```
class TableModel: public QAbstractTableModel {
```

Q-OBJECT

private:

std::vector<Obj> objects;

public:

explicit TableModel(const std::vector<Obj>& objects,
QObject* parent = nullptr)

int rowCount(---)

int columnCount(---)

QVariantData(---)

void updateData(const std::vector<Obj>& new);

Qt::ItemFlags flags(---) ---;

};

cpp

```
TableModel::TableModel(---): objects{objects}, QAbstractTableModel{  
parent} {}
```

```
--- rowCount(---) --- {
```

```
return static_cast<int>(objects.size());
```

```
}
```

```
--- columnCount(---) --- {
```

```
return 5;
```

```
}
```

```
--- data(---) --- {
```

```
if (!index.isValid() || role != Qt::DisplayRole)
```

```
return QVariant{};
```

```
const auto& o = objects[index.row()];
```

```
if (index.column() == 0)
```

```
return QString::from---
```

```
;
```

```
if (index.column() == 4)
```

```
return o.get(---);
```

```
return QVariant{};
```

```
}
```

```
--- updateData(---) --- {
```

```
beginResetModel();
```

```
objects = new---
```

```
endResetModel();
```

```
}
```

```
--- flags(---) --- {
```

```
return Qt::ItemIsSelectable | Qt::ItemIsEnabled |  
Qt::ItemIsEditable;
```

(review star app)



• inside GUI.h: ✓

as a field: TableModel* model

• inside GUI.cpp: constructor

model = new TableModel { service.get...(), this };
ui → tableView → setModel(model);

• Selections:

QModelIndex selection = ui → tableView → selectionModel() → selectedIndexes();
if (selection.empty())
return;

if inside service I have
const auto& stars = service.get...();
std::vector<Star> stars;

int row = selection.at(0).row()

I. const auto& stars = service.get...();
const Star& star = stars[row]

if no sorting/order

auto stars = service.get...();
Star selectedStar = stars[row]

checkedBox etc.
if sorted/order

if sorted/order/checked
currently displayed(---)
OR

model → data

inside populateTable()

current = service.getStars();
model → clear();
for...

• inside update():

model → updateData();

6) Heavy parsing

• list of offers: <userID, date, offered-sum>

• load function(hope)

```
std::getline(ss, offersString);  
std::vector<std::tuple<int, std::string, int>> offers;  
std::stringstream sss(offersString);  
std::string offer;  
while (std::getline(sss, offer, ';')) {  
    std::stringstream ssss(offer);  
    std::string id, date, sum;  
    std::getline(ssss, id, '|');  
    std::getline(ssss, sum, '|');  
    int ... = stoi(...);  
    offers.push_back(std::make_tuple(id, date, sum));  
}
```

offers[0] offers[1]
userID|date|sum; userID|date|sum

- save function (repo):

```

for (const i : items) {
    fout << i.get... () << " , "
    << i.get... () << " , "
    << i.get... () << " , " ;
    auto offers = i.getList... ();
    for (int i = 0; i < offers.size(); i++) {
        fout << std::get<0>(offers[i]) << " | "
        << std::get<1>(offers[i]) << " | "
        << std::get<2>(offers[i]) << " | "
        if (i != offers.size() - 1)
            fout << " , " ;
    }
    fout << " \n " ;
}
fout.close();

```

- string to vector of strings (load repo)

```

std::getline(ss, interestslist, '|');
std::vector<std::string> interests;
std::stringstream sss(interestslist);
std::string interest;
while (std::getline(sss, interest, '|')) {
    if (!interest.empty()) {
        interests.push_back(interest);
    }
}
volunteers.replace_back(..., interests, department);

```

(save repo)

```

for (...) {
    fout << n.get... () << " | " ;
    fout << r.get... () << " | " ;
    auto interests = ... ;
    for (int i = 0; i < interests.size(); i++) {
        fout << interests[i] ;
        if (i != interests.size() - 1) {
            fout << " , " ;
        }
    }
    fout << " | " ; fout << n.get... () ; fout << " \n " ;
}

```


7) save to files automatically.

```
std::string filename = writer.getName() + "_" + std::to_string(...) + ... + ".txt";
std::ofstream fout(filename);
...
fout << textEdit->toPlainText().toString();
fout.close();
...
```

8) save in the context of receiving arg etc. from expiring group...

inside the fct. → where I create the new window

```
connect(saveButton, &QPushButton::clicked, this, [this, textEdit, idea]() {
    saveDevIdea(textEdit, idea);
});
```

```
void GUI::saveDevIdea(QTextEdit* textEdit, const Idea& idea) {
```

```
}
```

8) update (validate) only when two others validated

• inside repo:

```
std::map<std::string, std::vector<std::string>> validations;
```

• update inside repo!

```
for (auto &d: drivers) {
```

```
    if (d.getName() == ...) {
```

```
        if (...)
```

```
            d.set...();
```

```
    }
}
```


8) QPainter

h

→

public

void update() override; ← repaint();

protected:

void PaintEvent(--);

private

Service & service;

cpp

→

--- paintEvent --- {

QPainter painter(this);

auto...

int startX = 220;

int y = 40;

for(---) {

painter.setBrush(Qt::green);

painter.setPen(Qt::green);

painter.drawPoint(x, y);

painter.drawText(x, y, QString::from

painter.drawEllipse(startX - radius,

y - radius, 2 * radius, 2 * radius);

painter.drawRect(x, y, w, h);

} y += 80;

- draw a rectangle of the same size as the window.
painter.drawRect(this->rect());

8) dynamic search (while typing)

- inside connect signals and slots . . . :

```
connect(ui → QLineEdit, &QLineEdit::textChanged, this, ::set)
```

- inside function set

```
ui → listWidget → clear();
```

```
std::string text = ui → QLineEdit → text().toStdString();
```

```
if (text.empty())
```

```
    return;
```

```
auto . . .
```

```
for ( . . . ) {
```

```
    ui → listWidget → addItem( . . . )
```

- inside service → find matching text:

```
for ( . . . ) {
```

```
    if ( s.getName().find(text) != std::string::npos)
```

```
        result.push_back(s);
```

```
} }
```

9) change font:

```
QFont font = item → font();
```

```
font.setBold(true);
```

```
item → setFont(font);
```


1) QSliders

```
connect(ui->slider, &QSlider::valueChanged, this, &GUI::radiusChanged)
connect(ui->slider, &QSlider::valueChanged, this, &GUI::radiusCh...)
connect(ui->slider, &QSlider::valueChanged, this, &GUI::radiusCh...)
void GUI::radiusChanged(int val) {
    radius = val;
    populateList();
}
```

```
connect(ui->slider, &QSlider::valueChanged, this, &GUI::radiusCh...)
void GUI::radiusChanged(int val) {
    radius = val;
    populateList();
}
```

2) Open new Window

```
QWidget* window = new QWidget();
```

→ type in:

```
auto layout = new QVBoxLayout(window);
for( ... ) {
```

```
    auto textEdit = new QTextEdit();
```

```
    textEdit->setText(QString::fromStdString(...));
```

```
    auto button = new QPushButton("save");
```

```
    layout->addWidget(textEdit);
```

```
    layout->addWidget(button);
```

```
    connect(button, &QPushButton::clicked, this,
        [this, textEdit, idea]() { save(...); });
```


3) TableModel

```
.h class TableModel: public QAbstractTableModel {
    Q_OBJECT
private:
    std::vector<...>
public:
    explicit TableModel (... vector ... & ...,
        QObject* parent = nullptr);
    int rowCount(...)
    int columnCount(...)
    QVariant data(...)
    void updateData (... & newData);
};
```

```
.cpp TableModel::TableModel(...) : ... {
    QAbstractTableModel { parent } {}
    ... rowCount ...
    return static_cast<int>(objects.size());
    ... columnCount ...
    return 5;
    ... data ... {
        if (!index.isValid() || role != Qt::DisplayRole)
            return QVariant{};
        const auto & o = objects[index.row()];
        if (index.column() == 0)
            if ...
            return QVariant{};
    }
    ... updateData ... {
        beginResetModel(); objects = newData;
        endResetModel();
    }
}
```


1) Table model.h

```
class TableModel: public QAbstractTableModel {
    Q_OBJECT
private:
    std::vector<Object> objects;
public:
    // explicit TableModel ( ... &objects,
    // QObject* parent = nullptr );
    int rowCount ( ... );
    int columnCount ( ... );
    QVariant data ( ... );
    void updateData ( ... &newData );
};
```

2) Table model.cpp

```
TableModel::TableModel ( ... ): objects { objects },
    QAbstractTableModel { parent } {}

int rowCount ( ... ) {
    return static_cast<int>(objects.size());
}

int columnCount ( ... ) {
    return 5;
}

QVariant data ( ... ) {
    // updateData ( ... ) {
    //     beginResetModel();
    //     objects = newData;
    //     endResetModel();
    // }

    if (!index.isValid() || role != Qt::DisplayRole)
        return QVariant();
    if (index.row() < 0 || index.column() < 0)
        return QVariant();
    if (index.column() == 0)
        return QVariant();
    return QVariant();
}
```

3) How to use TableModel in GUI

• public field:

TableModel* model;

• constructor:

model = new TableModel { service..., this };

ui->tableView->setModel(model);

• update()

newData

model->updateData(newData);

• public field:

TableModel* model;

• constructor:

model = new TableModel { service..., this };

ui->tableView->setModel(model);

• GUI::update()

model->updateData(service...);

4) QPainter.h

public

void update() override

protected:

void paintEvent (...)

private:

service & service...

5) QPainter.cpp

```
... paintEvent ( ... ) {
    QPainter painter ( this );
    int startx = 220;
    int y = 40;
    for ( ... ) {
        painter.setBrush ( Qt::color );
        painter.setPen ( ... );
        painter.drawText ( x, y, string... );
        painter.drawPoint ( x, y );
        painter.drawEllipse ( x, y, 2*rad, 2*rad );
    }
    painter.drawRect ( ... );
    y += 80;
    ...
}
```

6) Selections with table model

QModelIndexList selection = ui->tableView->

selectionModel()->selectedIndexes();

if (selection.empty()) return;

int row = selection.at(0).row();

If not sorted / checked boxes

auto ideas = service...
Idea selIdea = ideas[row];

If sorted / checked...
-> get currently displayed (depends on checkbox / ComboBox ...)

7) Edit directly in cell of table model

• inside TableModel.cpp: FLAGS.

... flags() ... {

return Qt::ItemIsSelectable;

Qt::ItemIsEnabled;

Qt::ItemIsEditable;

8) Dynamic Search

connect(ui->lineEdit, &QLineEdit::textChanged)

function

```
ui->...->clear();
std::string text = ui->lineEdit->text();
std::string();
if (empty). return;
for (--)
    ui->addItem(--);
```

service

```
for (--) {
    if (s.getName().find(text) !=
        std::string::npos)
        res.push_back(s);
}
// Parse list string -> vector
std::getline(ss, interestList, '|');
std::vector<std::string> interests;
std::stringstream sss(interestList);
std::string interest;
while (std::getline(sss, interest, '|')) {
    if (!interest.empty()) {
        interests.push_back(interest);
    }
}
```

10) Change font

```
QFont font = item->font();
font.setBold(true);
item->setFont(font);
```

11) Selection with list

```
auto selection = ui->listWidget->
    selectedItems();
if (selection.empty()) return;
int index = ui->listWidget->
    currentRow();
```

```
1) auto items = service.get
    Item item = items[index]
```

```
2) std::string item = selection[0]->text();
    toString();
```

3) populate listWidget + change background of sth.

```
ui->listWidget->clear();
for (--) {
    auto item = new QListWidgetItem
        (QString(...))
    ui->listWidget->addItem(item);
    item->setBackground(...)
```

13) QSlider connect + fct.

connect(ui->slider, &QSlider::valueChanged, this->GUI::radiusChanged)

```
void GUI::radiusChanged(int value) {
    radius = value; // radius -> public field.
    populateList();
    ui->slider->value(); // retrieve value of a slider.
```

14) Create new window from GUI, set a layout for it, set a textBox and a button

```
QWidget* window = new QWidget();
auto layout = new QVBoxLayout(window);
auto textEdit = new QTextEdit();
textEdit->setText(...);
auto button = new QPushButton("save");
layout->addWidget(textEdit);
layout->addWidget(button);
window->show
```

15) get currently displayed items [GUI]

CheckBox

```
if (ui->checkBox->isChecked())
    return service.getFiltered();
return service.getAll();
```

ComboBox

```
std::vector<...> result;
QString selected = ui->comboBox->currentText();
for (e in getAll()) {
    if (selected == "All")
        QString::fromStdString(e.toString()) =
            selected; result.push_back(e);
    return result;
}
if (selected == "All") return service.getAll();
return service.get(...);
```

16) parse: userID/date/sum; userID/date/sum;

17) store tuples (vector of tuples) → load (repo)

18) work with tuples

19) save fct. (vector of strings → file).

```
for(---) {  
    fct << ...  
    auto interests = ...  
    for(int i=0; i<interests.size(); i++) {  
        fct << interests[i];  
        if(i != interests.size()-1)  
            fct << ", ";  
    }  
}
```

20) save to file automatically

```
std::string filename;  
std::ofstream fct(filename);  
fct << textEdit->toPlainText().toStdString();
```

21) draw rectangle of the same size as window

painter → drawRect(this->rect());

22) get current date.

```
std::string currentDate =  
QDate::currentDate().toString("yyyy-MM-dd")  
    .toStdString();
```

23) find matching text (not exact match)



24) checkBox get + connect

↳ toggled



25) ComboBox constructor

```
ui->comboBox->addItem("All");  
for(---) {  
    ui->comboBox->addItem(---)
```

26) ComboBox connect

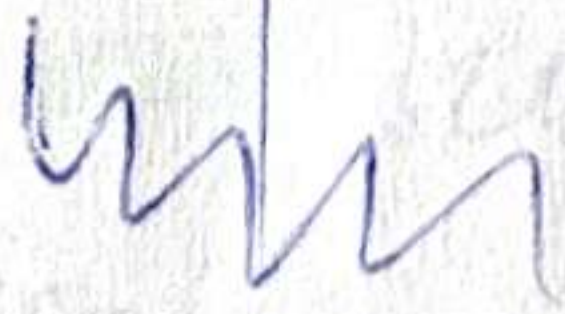
```
connect(ui->comboBox, &QComboBox::  
currentTextChanged, this, ---)
```

27) ComboBox retrieve info from it.

```
std::string selected = ui->comboBox->  
    currentText().toStdString();  
for(---) {  
    if (category == "All" || ... == category)  
        result.push_back(---);  
}
```

28) retrieve info from a lineEdit

```
ui->lineEdit  
std::string info = ui->lineEdit->  
    text().toStdString();  
int mr = ui->lineEdit->text().toInt();
```



29) put info on a list widget
(faster way)

for(....)
ui->listWidget->addItem(QString::....)

30) unpack Tuples

31) put info on a label
ui->label->setText(QString::....)

32) MessageBox for a good thing
(not for an error)

QMessageBox::information(this, "Success",
"Successful add");

33) select an item from a listWidget
and retrieve ~~the~~ the text of that
- the object } item.
- the index }

auto selection = ui->listWidget->selectedItems();
if (selection.empty())
return;

text ↗
std::string equation = selection[0]->text().
toStdString();

object
int index = ~~ui~~ ui->listWidget->currentRow();
auto equations = service.

+ connect
NOT NEEDED FOR
NORMAL SELECTION!

34) populate list with a vector of strings.

35) set color of the window,

36) get "top 3"

→ DEFAULT CONSTRUCTOR!
Volunteer() = default;
if (topVol.size() > 3)
topVol.resize(3);

37) count nr. of words in a string
std::stringstream(description);
std::string word;
int count = 0;
while (ss >> word) {
count++;
}
return count;

38) delete a given object
for (auto it = issues.begin();
it != issues.end(); ++it) {
if (~~it->getOwner()~~ ~~is~~ ~~the~~ ~~same~~ ~~as~~ ~~the~~ ~~one~~ ~~to~~ ~~be~~ ~~deleted~~ ~~...~~)
~~this~~ issues.erase(it);
}

39) if I want my buttons to
update (enable / disable) when I
select sth. from ...

A LIST: connect(ui->listWidget,
&QListWidget::itemSelectionChanged,
this, &GUI::updateButton);

A TABLE: connect(ui->tableView->
selectionModel(), &QItemSelectionModel::selection
Changed, ...)